

**LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 3 ACARA 5
SEMESTER 3**



**Tugas :
PHP Native View, Layout, dan Dasar OOP**

**Disusun Oleh:
Uaryl Engno Laurenca (E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. PENDAHULUAN

Tanpa mekanisme routing, URL sebuah aplikasi web PHP native akan terlihat mengekspos struktur folder secara langsung, misalnya `http://localhost/si-akademik/app/Controllers/MahasiswaController.php?action=index`. Selain terlihat tidak profesional, URL semacam ini juga membuka celah keamanan karena struktur internal aplikasi menjadi terlihat oleh pengguna.

Routing mengatasi masalah ini dengan memetakan URL yang bersih (misalnya `/mahasiswa` atau `/mahasiswa/5`) ke method Controller yang sesuai, tanpa pengguna perlu tahu di file mana kode tersebut sebenarnya berada. Agar seluruh request diarahkan ke satu titik masuk (*front controller*), diperlukan konfigurasi `.htaccess` pada web server Apache untuk melakukan *URL rewriting*. Praktikum Acara 5 ini menerapkan kedua konsep tersebut sekaligus memahami superglobal PHP (`$_GET`, `$_POST`, `$_SERVER`) yang berperan dalam menangani request dan response.

B. TUJUAN PRAKTIKUM

1. Memahami konsep routing sebagai proses pemetaan URL ke Controller dan method yang tepat.
2. Membuat file `.htaccess` agar seluruh request diarahkan ke satu *entry point* (`public/index.php`).
3. Membuat `public/index.php` sebagai *front controller* yang membaca URL, mencocokkannya ke daftar routes, lalu memanggil Controller yang sesuai.
4. Memahami dan menggunakan superglobal PHP (`$_GET`, `$_POST`, `$_SERVER`) dalam konteks routing.
5. Menerapkan `header("Location: ...")` beserta `exit` untuk melakukan redirect dengan aman.
6. Menguji beberapa URL (termasuk URL yang tidak terdaftar) untuk memastikan routing dan penanganan 404 berjalan benar.
7. Menambahkan dukungan parameter URL dinamis (misalnya `/mahasiswa/5`) sebagai tugas mandiri.

C. DASAR TEORI

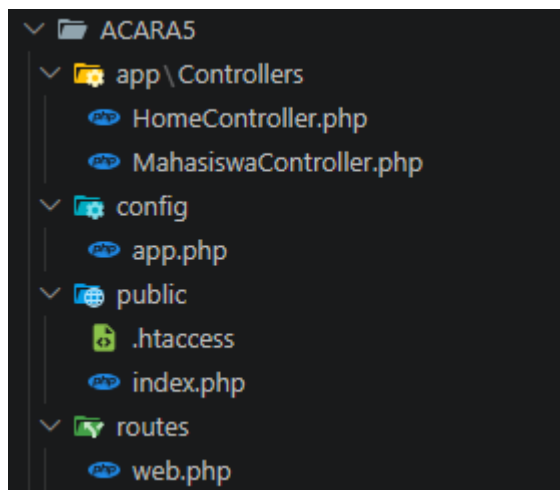
Routing adalah proses memetakan URL request ke kode yang sesuai, biasanya berupa method pada Controller. Dalam praktikum, routing dibuat menggunakan array asosiatif yang mengelompokkan rute berdasarkan method HTTP seperti `GET` dan `POST`, kemudian setiap URL diarahkan ke Controller dan method yang sesuai. `.htaccess` digunakan untuk mengatur URL rewriting pada Apache agar request yang tidak mengarah ke file atau folder yang tersedia diteruskan ke `index.php` sebagai entry point. Seluruh request kemudian diproses melalui `public/index.php`, yang mengambil URL dan method HTTP, mencocokkannya dengan routes, lalu memanggil Controller yang sesuai atau menampilkan halaman 404 jika rute tidak ditemukan. Dalam proses tersebut digunakan beberapa superglobal PHP, seperti `$_GET` untuk query string, `$_POST` untuk data form, dan `$_SERVER` untuk mendapatkan informasi request seperti `REQUEST_URI` dan `REQUEST_METHOD`. Selain itu, redirect dapat dilakukan menggunakan

`header("Location: {url}")` yang umumnya menghasilkan status HTTP 302, kemudian harus diikuti `exit` atau `die` agar eksekusi kode berhenti setelah proses redirect.

D. HASIL DAN PEMBAHASAN

1. Struktur Folder Project

Dibuat folder project baru `ACARA5` dengan struktur berikut:



2. Pembuatan `routes/web.php`

```
routes > web.php
1  <?php
2
3  $routes = [
4      'GET' => [
5          '/' => ['HomeController', 'index'],
6          '/mahasiswa' => ['MahasiswaController', 'index'],
7          '/mahasiswa/create' => ['MahasiswaController', 'create'],
8          '/mahasiswa/{id}' => ['MahasiswaController', 'show'], // Tugas Mandiri
9          '/login' => ['AuthController', 'loginForm'],
10     ],
11     'POST' => [
12         '/mahasiswa' => ['MahasiswaController', 'store'],
13         '/login' => ['AuthController', 'login'],
14     ],
15 ];
16
17 return $routes;
```

Dibuat array `$routes` yang mengelompokkan rute berdasarkan method HTTP, memetakan setiap URL ke pasangan Controller dan method-nya, termasuk satu rute tambahan `/mahasiswa/{id}` untuk kebutuhan tugas mandiri (parameter dinamis).

3. Pembuatan public/.htaccess

```
public >  .htaccess
1 RewriteEngine On
2 RewriteCond %{REQUEST_FILENAME} !-f
3 RewriteCond %{REQUEST_FILENAME} !-d
4 RewriteRule ^ index.php [QSA,L]
```

Dibuat file .htaccess di folder public agar seluruh request yang bukan file/folder nyata diteruskan ke index.php, sesuai contoh pada modul.

4. Pembuatan public/index.php sebagai Front Controller

```
1 <?php
2
3 require_once __DIR__ . '/../config/app.php';
4
5 // Autoloader sederhana untuk namespace App\...
6 spl_autoload_register(function ($class) {
7     $class = str_replace('App\\', '', $class);
8     $file = __DIR__ . '/../app/' . str_replace('\\', '/', $class) . '.php';
9     if (file_exists($file)) {
10         require_once $file;
11     }
12 });
13
14 $routes = require __DIR__ . '/../routes/web.php';
15
16 $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
17
18 // Hilangkan base path karena project ada di subfolder
19 $base = '/BKPM-Web-Server/ACARA5/public';
20 if (str_starts_with($uri, $base)) {
21     $uri = substr($uri, strlen($base)) ?: '/';
22 }
23 $uri = rtrim($uri, '/') ?: '/';
24
25 $method = $_SERVER['REQUEST_METHOD'];
26
27 // 1. Coba exact match dulu (untuk URL statis seperti /mahasiswa)
28 if (isset($routes[$method][$uri])) {
29     [$controllerName, $action] = $routes[$method][$uri];
30     callAction($controllerName, $action);
31     exit;
32 }
33
34 // 2. Kalau tidak ketemu, coba cocokkan pola dinamis seperti /mahasiswa/{id}
35 foreach ($routes[$method] ?? [] as $pattern => $handler) {
36     if (!str_contains($pattern, '{')) {
37         continue;
38     }
39
40     $regex = preg_replace('#\[a-zA-Z_+\]#', '([^/]+)', $pattern);
41     $regex = '#^' . $regex . '$#';
42
43     if (preg_match($regex, $uri, $matches)) {
44         array_shift($matches); // buang hasil full match, sisakan parameter saja
45         [$controllerName, $action] = $handler;
46         callAction($controllerName, $action, $matches);
47         exit;
48     }
49 }
50
51 // 3. Tidak ketemu sama sekali -> 404
52 http_response_code(404);
53 echo "<h1>404 - Halaman Tidak Ditemukan</h1>";
54 echo "<p>URL: " . htmlspecialchars($uri) . "</p>";
55 exit;
56
57 function callAction(string $controllerName, string $action, array $params = []): void
58 {
59     $controllerClass = "App\\Controllers\\" . $controllerName;
60
61     if (!class_exists($controllerClass)) {
62         http_response_code(500);
63         echo "Controller {$controllerClass} tidak ditemukan.";
64         return;
65     }
66
67     $controller = new $controllerClass();
68     call_user_func_array([$controller, $action], $params);
69 }
```

index.php bertugas mengambil URI dari `$_SERVER['REQUEST_URI']`, menghilangkan base path (`/ACARA5/public`) agar sesuai dengan key pada array routes, lalu mencocokkannya berdasarkan method HTTP. Apabila URL cocok persis dengan salah satu key di routes, Controller dan method terkait langsung dipanggil. Apabila tidak ditemukan kecocokan persis, sistem mencoba mencocokkan pola dinamis (`{id}`) menggunakan regex sebelum akhirnya menampilkan halaman 404 jika benar-benar tidak ditemukan.

5. Pembuatan HomeController dan MahasiswaController

```
app > Controllers > HomeController.php
1  <?php
2
3  namespace App\Controllers;
4
5  class HomeController
6  {
7      public function index()
8      {
9          echo "<h1>Selamat Datang</h1>";
10         echo "<p>Ini adalah HomeController::index()</p>";
11     }
12 }
```

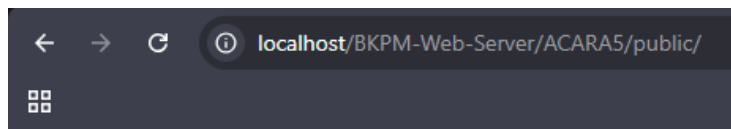
```
app > Controllers > MahasiswaController.php
1  <?php
2
3  namespace App\Controllers;
4
5  class MahasiswaController
6  {
7      public function index()
8      {
9          echo "<h1>Daftar Mahasiswa</h1>";
10         echo "<p>Ini adalah MahasiswaController::index()</p>";
11     }
12
13     public function create()
14     {
15         echo "<h1>Form Tambah Mahasiswa</h1>";
16         echo "<p>Ini adalah MahasiswaController::create()</p>";
17     }
18
19     public function store()
20     {
21         echo "<h1>Menyimpan Data Mahasiswa (POST)</h1>";
22         echo "<p>Ini adalah MahasiswaController::store()</p>";
23     }
24
25     // Tugas Mandiri: menerima parameter dari URL, misal /mahasiswa/5
26     public function show($id)
27     {
28         echo "<h1>Detail Mahasiswa</h1>";
29         echo "<p>Menampilkan mahasiswa dengan ID: " . htmlspecialchars($id) . "</p>";
30     }
31 }
```

Dibuat dua Controller sederhana yang masing-masing method-nya hanya menampilkan echo sesuai instruksi modul, sebagai bukti bahwa routing berhasil mengarahkan request ke method yang tepat.

6. Pengujian Routing

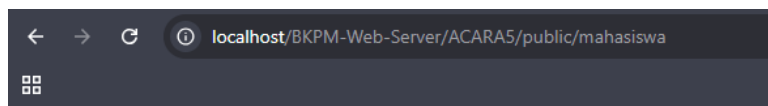
Dilakukan pengujian terhadap beberapa URL untuk memastikan routing dan `.htaccess` berfungsi dengan benar:

URL yang diuji	Hasil
<code>http://localhost/BKPM-Web-Server/ACARA5/public/</code>	Menampilkan "Selamat Datang" dari <code>HomeController::index()</code>
<code>http://localhost/BKPM-Web-Server/ACARA5/public/mahasiswa</code>	Menampilkan "Daftar Mahasiswa" dari <code>MahasiswaController::index()</code>
<code>http://localhost/BKPM-Web-Server/ACARA5/public/mahasiswa/create</code>	Menampilkan "Form Tambah Mahasiswa" dari <code>MahasiswaController::create()</code>
<code>http://localhost/BKPM-Web-Server/ACARA5/public/halaman-random</code>	Menampilkan halaman 404 buatan sendiri



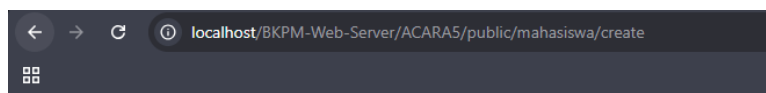
Selamat Datang

Ini adalah `HomeController::index()`



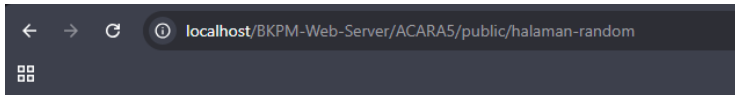
Daftar Mahasiswa

Ini adalah `MahasiswaController::index()`



Form Tambah Mahasiswa

Ini adalah `MahasiswaController::create()`



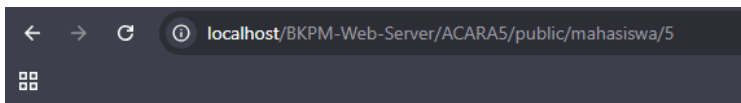
404 - Halaman Tidak Ditemukan

URL: /halaman-random

Seluruh URL bersih (tanpa ekstensi `.php` maupun path folder internal) berhasil diproses dengan benar oleh router, membuktikan bahwa `.htaccess` dan *front controller* bekerja sebagaimana mestinya.

7. Tugas Mandiri Parameter URL Dinamis

Ditambahkan dukungan parameter URL dengan mendefinisikan rute berpola `/mahasiswa/{id}` pada `routes/web.php`, serta method `show($id)` pada `MahasiswaController`. Pada `index.php`, apabila pencocokan persis (*exact match*) tidak ditemukan, sistem mengubah pola rute (misalnya `/mahasiswa/{id}`) menjadi ekspresi reguler `#^/mahasiswa/([^\/]*)$`, kemudian mencocokkannya dengan URL yang diminta menggunakan `preg_match()`. Apabila cocok, nilai yang tertangkap (misalnya `5` dari URL `/mahasiswa/5`) diteruskan sebagai argumen ke method `show($id)`.



Detail Mahasiswa

Menampilkan mahasiswa dengan ID: 5

Pengujian dengan mengakses `http://localhost/BKPM-Web-Server/ACARA5/public/mahasiswa/5` berhasil menampilkan "Detail Mahasiswa" dengan ID: 5, membuktikan bahwa parameter dari URL berhasil diteruskan dan diterima oleh Controller.

E. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa routing memungkinkan pemisahan antara URL yang digunakan pengguna dengan struktur folder atau file aplikasi sehingga menghasilkan URL yang lebih bersih dan terstruktur. Konfigurasi `.htaccess` dengan `mod_rewrite` berperan dalam mengarahkan request ke satu *entry point*, yaitu `index.php`, sehingga proses routing dapat ditangani oleh aplikasi PHP. `public/index.php` sebagai *front controller* berhasil membaca URL dan method HTTP melalui `$_SERVER`, mencocokkannya dengan routes, kemudian memanggil Controller dan method yang sesuai. Penanganan URL yang tidak terdaftar

menggunakan `http_response_code(404)` juga memberikan respons yang jelas ketika halaman tidak ditemukan. Selain itu, penerapan parameter URL dinamis seperti `/mahasiswa/{id}` menunjukkan bahwa routing dapat dikembangkan untuk menangani data seperti ID pada proses detail, edit, atau delete. Pemahaman terhadap superglobal `$_GET`, `$_POST`, dan `$_SERVER`, serta penggunaan `exit` setelah `header("Location: ...")`, menjadi dasar penting dalam pengembangan form dan proses redirect pada aplikasi PHP.